



UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

ARTIFICIAL INTELLIGENCE II

AI for Robotics II Project

Modeling an automated warehouse scenario

Author:

Lisa Mokrani
Mohamed Aguenarous
Chaima Ferdjallah

Student ID:

s7229301
s6752572
s5864751

Professors:

Mauro Vallati
Matteo Cardellini

May 22, 2025

Contents

1	Introduction	3
1.1	Context and Motivation	3
1.2	Project Description	3
1.3	Project Goals	3
1.4	Technical Challenges	3
2	Domain models	4
2.1	Domain Assumptions	4
2.2	Types and Objects	4
2.3	Predicates and Functions	5
2.3.1	Predicates	5
2.3.2	Functions	6
2.4	Actions	7
2.4.1	Movement Actions	7
2.4.2	Crate Transportation	7
2.4.3	Loading and Recharging	7
2.4.4	Particular case to solve problem 3	8
2.5	Domain Extensions	8
2.5.1	Crate Groups and Loading Sequence	8
2.5.2	Secondary Loader for Light Crates	8
2.5.3	Battery Management for Robots	9
2.5.4	Handling Fragile Crates	9
3	Planning and Evaluation	9
3.1	Temporal Planning	9
3.2	Experimental Setup	10
3.2.1	Software Configuration	10
3.2.2	Execution Methodology	11
3.2.3	Planner Overview: Temporal Fast Downward (TFD)	11
3.2.4	Performance Metrics	11
3.3	Problem Scenarios	11
3.3.1	Problem 0.5 - Simple Scenario	11
3.3.2	Problem 1 - Medium Scenario (Fragile Crates)	12
3.3.3	Problem 2 - Complex Scenario (Multi-Group and Battery Management)	12
3.3.4	Problem 3 - Complex Scenario (Unreachable Crate Due to Battery Constraints)	12
3.3.5	Problem 4 - Advanced Scenario (Multiple Groups and Fragile Crates)	12
4	Heuristic and Pattern Design	14
4.1	Heuristic Function Design	14
4.1.1	Heuristic Objective	14
4.1.2	Pseudocode	14
4.1.3	Heuristic Description	14
4.1.4	Worked Example: Problem 0.5	14
4.1.5	Why the Heuristic is Useful	15
4.1.6	Admissibility	15
4.1.7	Heuristic vs. Actual Plan Comparison	15
4.1.8	Conclusion	15
4.2	Pattern Definition and Symbolic Bounds	15
4.2.1	Pattern for Problem 0.5	15
4.2.2	Symbolic Pattern Bound for Problem 0.5	16
4.2.3	Behavior in Larger Problems	16
4.2.4	Generalizing the Pattern	16
5	Conclusion	17
5.1	Experimental Results	17
5.1.1	Problem 0.5	17
5.1.2	Problem 1	17
5.1.3	Problem 2	18
5.1.4	Problem 3	19

5.1.5	Problem 4	19
5.1.6	Summary of Results	20
5.2	Challenges Faced	21
5.2.1	Battery Constraints and Inaccessible Crates (Problem 3)	21
5.2.2	Loader Role Separation and Balanced Task Distribution	21
5.3	Possible Improvements	22
5.3.1	Generalizing Relay Strategy for All Crates	22

1 Introduction

1.1 Context and Motivation

- In modern logistics and e-commerce, automation has become essential to manage the increasing volume and complexity of warehouse operations. The use of autonomous robots in warehouses enables more efficient handling of storage and delivery tasks, particularly when systems must account for constraints such as energy usage, load capacity, and delivery priorities...
- Planning-based robotic control is especially effective in such scenarios. Using AI planning frameworks like PDDL (Planning Domain Definition Language), warehouse environments can be modeled formally, allowing autonomous agents (robots) to reason about their actions, battery limitations, delivery sequences, and shared resources such as the loading bay.
- This project investigates how such models can be constructed and tested under realistic constraints, with a specific focus on how robots transport crates while operating under battery limitations and various task dependencies.

1.2 Project Description

The project centers around a simulated warehouse environment in which robots are tasked with transporting crates from remote storage locations to a central conveyor belt. Each crate may possess unique properties:

- **Weight:** Crates may be classified as heavy or light, with heavy crates requiring cooperative transport by two robots.
- **Fragility:** Fragile crates must be handled with care and are loaded differently.
- **Ordering:** Some crates must be delivered in a specified order, as determined by their group assignment.
- **Robots operate under strict energy constraints:** Each robot has a battery level that decreases with movement or task execution. Recharge is only available at the loading bay, making planning non-trivial.

In addition, the loading bay supports two loading arms (primary and secondary), but only the primary can handle heavy crates. Coordination between multiple robots and avoidance of concurrent loading conflicts is essential.

1.3 Project Goals

The main goals of the project are:

1. To design a robust and realistic PDDL domain model that reflects all constraints above.
2. To support diverse problem instances, from simple crate transportation to complex coordinated delivery with energy and ordering constraints.
3. To enable fallback behavior when not all crates can be delivered due to battery limits (e.g., skipping unreachable crates).
4. To evaluate the performance of planning strategies in both solvable and infeasible instances.

1.4 Technical Challenges

Several technical challenges were encountered:

- Modeling numeric fluents (battery level, total time) alongside durative actions
- Enforcing delivery orderings without direct time constraints
- Supporting dynamic recharging logic and conditional movement actions
- Preventing physically implausible actions, such as moving heavy crates alone or reloading crates already loaded

The domain was designed to be compatible with automated planners like TFD (Temporal Fast Downward), and tested using a variety of benchmark problems with increasing complexity.

2 Domain models

2.1 Domain Assumptions

The warehouse domain model is designed to simulate a warehouse environment where autonomous robots transport crates between locations, including a loading bay. The model is built with the following assumptions:

1. Robot Capacity:

- Robots can carry one crate at a time.
- Heavy crates require two robots to transport, while light crates can be handled by a single or two robots.

2. Energy Management:

- Robots have limited battery capacity (max = 20 units) and must recharge periodically in the loading bay .
- The battery consumption is proportional to the distance traveled and the weight of the transported crate.

3. Crate Properties:

- Crates are categorized as heavy, fragile, or light.
- Fragile crates require careful handling and are loaded using special actions.
- Grouped crates must be loaded in a specific order.

4. Location Management:

- The warehouse has multiple locations, including a designated loading bay.
- Robots can freely move between locations unless blocked by logical constraints.

5. Concurrent Operations:

- Multiple robots can operate simultaneously.
- Actions are durative (take time to complete), and their durations depend on factors like distance and crate weight.

6. Load Logic : The loading process is managed by two types of loaders:

- Primary Loader: Handles light ,heavy and fragile crates.
- Secondary Loader: s limited to handling light crates only, and cannot load heavy crates, but it can load both fragile and non-fragile light crates.

2.2 Types and Objects

1. Types :

```
(:types
  robot crate location group
)
```

- **robot**: Represents the autonomous robots.
- **crate**: Represents the crates that need to be transported.
- **location**: Represents all possible locations in the warehouse, including the loading bay.
- **group**: Represents crate groups for sequential loading.

2. Objects :

```
(:objects
  robot1 robot2 - robot
  crate1 crate2 crate3 crate4 - crate
  loc10 loc20 loc30 - location
  loading-bay - location
  groupA - group
)
```

2.3 Predicates and Functions

This section provides a detailed description of the predicates and functions used in the warehouse domain model. Predicates define the logical state of objects within the domain, while functions provide quantitative measures that influence action conditions and effects.

2.3.1 Predicates

```
(at ?r - robot ?l - location)
```

Indicates that a specific robot (*?r*) is located at a particular location (*?l*). It is used in movement actions to ensure a robot is at the correct starting location before an action can begin.

Example: (at robot1 loc10) means *robot1* is currently at location *loc10*.

```
(crate-at ?c - crate ?l - location)
```

Specifies that a crate (*?c*) is located at a certain location (*?l*). This predicate is critical for all transportation actions where crates are moved between locations.

Example: (crate-at crate1 loading-bay) means *crate1* is currently in the loading bay.

```
(crate-loaded ?c - crate)
```

Indicates that a crate (*?c*) has been successfully loaded (i.e., it is no longer at any location).

Example: (crate-loaded crate2) means *crate2* has been loaded and is no longer in any location.

```
(is-heavy ?c - crate)
```

Identifies a crate as being heavy, requiring two robots for transportation.

Example: (is-heavy crate3) means *crate3* is classified as heavy.

```
(is-fragile ?c - crate)
```

Indicates that a crate is fragile and must be handled with care.

Example: (is-fragile crate4) means *crate4* is fragile.

```
(currently-in-loading-bay ?r - robot)
```

Indicates that robot *?r* is currently in the loading bay and available for actions like charging or loading co-ordination.

Example: (currently-in-loading-bay robot2) means *robot2* has arrived and is registered as present in the loading bay.

```
(is-loading-bay ?l - location)
```

Marks the specified location `?l` as a loading bay. Used to restrict actions to occur only at valid loading areas.

Example: `(is-loading-bay loc3)` asserts that *loc3* is a loading bay.

```
(has-arrived-at ?r - robot ?to - location)
```

Indicates that robot `?r` has arrived at location `?to`, confirming the effect of a move action.

Example: `(has-arrived-at robot1 loc2)` shows *robot1* just reached *loc2*.

```
(is-midpoint ?l - location)
```

Declares that a location `?l` serves as a midpoint—used in relay movement strategies for heavy crates.

Example: `(is-midpoint loc15)` indicates *loc15* is a valid relay midpoint.

```
(is-relay-required ?c - crate)
```

Used to flag crates that require a relay process (via midpoint) due to long distances or complex logistics.

Example: `(is-relay-required crate4)` shows that *crate4* should be handled through a relay approach.

2.3.2 Functions

```
(weight ?c - crate)
```

A numeric function representing the weight of a crate (`?c`). It directly impacts the duration of movement actions and the selection of appropriate transportation actions.

Example: If `(= (weight crate1) 50)`, then *crate1* weighs 50 units.

```
(distance-between ?from ?to)
```

Returns numeric distance between two locations.

Example: If `(= (distance-between loc10 loading-bay) 10)`, the distance between the location10 and the loading-bay is 10 units.

```
(battery-level ?r - robot)
```

A numeric function representing the current battery level of a robot (`?r`).

Example: If `(= (battery-level robot1) 20)`, then *robot1* has 20 units of battery.

```
(total-time)
```

A numeric function that accumulates the total time cost of the plan. It is increased manually inside each durative action to reflect the passage of time. This function is useful when using a metric such as :`metric minimize (total-time)` in the problem file to guide the planner toward faster (more efficient) plans.

Example: After several movement and loading actions that each include `(increase (total-time) 1)`, the value of `(total-time)` could reach 10, indicating a total accumulated cost of 10 time units in the plan.

2.4 Actions

2.4.1 Movement Actions

- **mark-ready-in-loading bay**
 - Marks a robot as ready and in the loading bay.
 - Instantaneous action.
- **move-empty-from-loading-bay**
 - Allows a free robot to move from the loading bay to a specified location if there is a crate at the destination.
 - The action duration is calculated as $(\text{distance} / 10)$.

2.4.2 Crate Transportation

- **move-crate-single**
 - Moves a single, non-heavy, non-fragile crate using one robot.
 - The duration is calculated as $(\text{distance} \times \text{weight}) / 100$.
 - Conditions ensure the robot is free, has enough battery, and the crate is not already loaded.
- **move-crate-light-duo**
 - Moves a light crate using two robots, allowing faster transport.
 - Duration is $(\text{distance} \times \text{weight}) / 150$.
 - Both robots must be free and present at the source.
- **move-crate-duo**
 - Moves a heavy crate using two robots.
 - The duration is $(\text{distance} \times \text{weight}) / 100$.
 - Both robots must be free, and the crate must be marked as heavy.

2.4.3 Loading and Recharging

- **load-crate-primary**

Loads a crate that is neither fragile nor light using the **primary loader**. It excludes fragile crates but includes any crate that is either heavy or simply requires the primary path. The action:

 - Duration : 4 time units.
 - Requires the crate to be at the loading bay and not already loaded.
 - Uses the `loading-in-progress-primary` predicate to prevent concurrent loading.
- **load-crate-fragile-primary**

Loads a **fragile crate** (regardless of weight) using the **primary loader**. This action is specifically reserved for careful handling:

 - Duration: 6 time units.
 - Ensures the crate is fragile.
 - Blocks other primary loader operations via `loading-in-progress-primary`.
- **load-crate-secondary**

Loads a crate that is **light and not fragile** using the **secondary loader**:

 - Duration: 4 time units.
 - Works only for crates that are `not (is-heavy)` and `not (is-fragile)`.
 - Prevents concurrent loading using `loading-in-progress-secondary`.
- **load-crate-fragile-secondary**

Loads a **fragile, light crate** using the secondary loader. This applies only when the crate is both fragile and not heavy:

 - Duration: 6 time units.
 - Requires both `(is-fragile ?c)` and `(not (is-heavy ?c))`.

- Exclusive use of `loading-in-progress-secondary`.
- **recharge**
Restores the robot's battery back to full (20 units), but only when the robot is present in the loading bay:
 - Duration: 5 time units.
 - Requires `(currently-in-loading-bay ?r)`.
 - Assigns battery level to 20 using `(assign (battery-level ?r) 20)`.

2.4.4 Particular case to solve problem 3

This subset of actions is tailored for situations where relay-based coordination is needed to transport heavy crates through an intermediate midpoint before reaching the final destination (e.g., the loading bay). These actions help overcome battery constraints and logistical separation by splitting movement into stages.

- **move-empty-to-loading-bay**
Allows a free robot to move from a midpoint location to a loading bay without carrying a crate. This is often used to prepare for the second stage of a relay operation.
- **move-empty-general**
Moves an unoccupied robot between general locations (non-loading bay), useful for coordinating robot availability in different zones.
- **move-crate-duo-relay**
Enables two robots to cooperatively carry a heavy crate from the origin to an intermediate (midpoint) location. This splits the full delivery into two phases for battery and distance efficiency.
- **move-crate-duo-to-final**
The final stage in the relay: two robots move the crate from the midpoint to the loading bay. This assumes the crate has already been partially transported to the midpoint.

2.5 Domain Extensions

In order to enhance the flexibility, realism, and efficiency of the warehouse domain model, several extensions have been incorporated. These extensions allow the model to capture complex behaviors and constraints commonly found in real-world scenarios. The following four extensions have been integrated:

2.5.1 Crate Groups and Loading Sequence

This extension introduces the concept of crate groups, where multiple crates are logically connected as a group. Crates within a group must be loaded in a specific sequence, ensuring that dependent crates are only loaded after their predecessors have been processed.

- `(crate-group ?c - crate ?g - group)`: Associates a crate with a specific group.
- `(group-next ?c1 - crate ?c2 - crate)`: Specifies that crate `?c2` can only be loaded after crate `?c1`.
- `(group-loaded ?g - group)`: Indicates that all crates in the group have been loaded.

2.5.2 Secondary Loader for Light Crates

The model includes a secondary loader mechanism designed for handling light crates. This loader can manage crates that are neither heavy nor fragile, reducing the workload of the primary loader.

- `(loading-in-progress-secondary)`: Ensures that only one secondary loading process is active at a time.

2.5.3 Battery Management for Robots

To increase the realism of the domain, robots are equipped with a battery management system. Each robot has a battery level that decreases as it performs actions, and they must periodically recharge.

- `(battery-level ?r - robot)`: A numeric function representing the current battery level of a robot.
- `recharge`: An action that allows a robot to recharge its battery, but only when in the loading bay.

2.5.4 Handling Fragile Crates

The domain model is extended to accommodate fragile crates, which require special handling procedures. Fragile crates are identified using the predicate:

- `(is-fragile ?c - crate)`: Indicates that a crate is fragile.

The handling of fragile crates affects both movement and loading actions:

- Fragile crates are loaded using specialized actions: `load-crate-fragile-primary` for primary loader and `load-crate-fragile-secondary` for secondary loader.
- Movement actions for fragile crates ensure careful handling, minimizing the risk of damage.

3 Planning and Evaluation

3.1 Temporal Planning

In our domain model we use Temporal planning model problems where actions take time and the timing matters. Formally, it can be represented as:

$$(V_B, V_N, A, I, G, C)$$

where:

- V_B (Boolean variables): represent facts that are true or false at a given time.
- V_N (Numerical variables): represent quantities that change over time, like fuel or temperature.
- A (Actions): each action has preconditions, duration, and effects on V_B and V_N .
- I (Initial state): values of all variables at the start.
- G (Goal): desired condition to reach.
- C (Constraints): temporal or resource constraints governing action scheduling.

Example: Robot Delivery Task

Imagine a robot delivering packages in a warehouse. We want to model the planning problem of the robot moving and delivering a package over time.

Boolean variables V_B :

- `at_location_A`: whether the robot is at location A (true/false)
- `has_package`: whether the robot currently holds the package

Numerical variables V_N :

- `battery_level`: battery charge percentage (0 to 100)
- `time`: current time (continuous)

Actions A :

1. Move from A to B

- Preconditions: `at_location_A = true`, `battery_level ≥ 10`
- Duration: 5 time units
- Effects:

- At start: $at_location_A \leftarrow \text{false}$
- At end: $at_location_B \leftarrow \text{true}$, $battery_level \leftarrow battery_level - 10$

2. Pick up package

- Preconditions: $at_location_B = \text{true}$, $has_package = \text{false}$
- Duration: 1 time unit
- Effects:
 - At end: $has_package \leftarrow \text{true}$

3. Deliver package

- Preconditions: $at_location_C = \text{true}$, $has_package = \text{true}$
- Duration: 2 time units
- Effects:
 - At end: $has_package \leftarrow \text{false}$, package delivered

Initial state I :

- $at_location_A = \text{true}$, $at_location_B = \text{false}$, $at_location_C = \text{false}$
- $has_package = \text{false}$
- $battery_level = 50$
- $time = 0$

Goal G :

- Package delivered (e.g., $has_package = \text{false}$ and delivery location reached)

Constraints C :

- Battery must never go below 0
- Actions must respect their durations and cannot overlap improperly

How Planning Happens Here

The planner must find a sequence and schedule of actions, such as:

1. Move from location A to B (takes 5 units, battery drops to 40)
2. Pick up package at B (1 unit)
3. Move from B to C (another action not fully detailed here, say it takes 7 units, battery drops to 20)
4. Deliver package at C (2 units)

3.2 Experimental Setup

The experiments were conducted using the **Temporal Fast Downward (TFD)** planner, a well-known temporal planner capable of handling durative actions and numeric fluents, which are essential for our warehouse domain. The planning domain and problem files were written in **PDDL (Planning Domain Definition Language)** using **Visual Studio Code (VS Code)** with the `PDDL extension` for syntax highlighting and debugging or PDDL Editor Website.

3.2.1 Software Configuration

- **Planner:** Temporal Fast Downward (TFD)
- **Programming Language:** PDDL (Planning Domain Definition Language)
- **Editing Environment:** Visual Studio Code (VS Code) with PDDL extension / PDDL Editor Website.
- **Execution Environment:** TFD executed within a *Singularity*¹ container for portability and consistency across environments.

¹Singularity is a container platform designed for portability, ensuring consistent execution across different environments.

3.2.2 Execution Methodology

The planner was executed using the following approach:

- The `domain.pddl` and `problem-x.pddl` files were defined according to the specifications of the warehouse domain model.
- Plans were generated using the TFD planner by specifying the domain and problem files as input.
- The planner's output was captured, including the action sequence and the makespan².
- The generated plans were evaluated based on their correctness (achieving the specified goal) and efficiency (minimizing makespan).

3.2.3 Planner Overview: Temporal Fast Downward (TFD)

Temporal Fast Downward (TFD) is an automated planner specialized in solving temporal planning problems. It receives a domain and problem definition in PDDL format and searches for a sequence of actions scheduled over time that lead from the initial state to the goal.

TFD employs heuristic search to efficiently explore possible plans while considering temporal aspects such as action durations and concurrency. This allows the planner to generate temporally optimized plans where actions can overlap if the problem permits.

The output is a temporal plan that specifies start times for each action, ensuring that all preconditions are satisfied, no conflicts occur, and the goals are achieved efficiently.

3.2.4 Performance Metrics

The following metrics were used to evaluate the planner's performance:

- **Makespan:** The total duration of the generated plan, calculated based on the temporal durations of the actions.
- **Plan Length:** The number of actions in the generated plan.
- **Computation Time:** The time taken by the planner to generate a valid plan.
- **Plan Validity:** Verification that the generated plan correctly satisfies the specified goal state.

3.3 Problem Scenarios

This section describes a series of problem scenarios defined for the warehouse domain, each introducing varying levels of complexity. These scenarios test the planner's ability to handle different aspects of the domain, including group management, fragile crates, battery management, and unreachable crates.

3.3.1 Problem 0.5 - Simple Scenario

- **Description:** A basic scenario with two crates, one heavy and one light, both needing to be loaded. The scenario tests basic movement and loading actions without complex conditions.
- **Key Aspects:**
 - Two crates: one heavy (`crate1`) and one light (`crate2`).
 - Both crates belong to the same group (Group A) with a specified loading order.
 - Both robots start with enough battery to complete their tasks.
- **Challenge:** Efficiently load the two crates while maintaining the specified group order.
- **Objective:** Load `crate1` followed by `crate2` onto the conveyor belt.

²total duration of the plan

3.3.2 Problem 1 - Medium Scenario (Fragile Crates)

- **Description:** This scenario adds complexity by introducing a fragile crate that requires careful handling during loading.
- **Key Aspects:**
 - Three crates: one heavy (crate1), one fragile (crate2), and one standard (crate3).
 - Crate2 is marked as fragile, necessitating careful loading.
 - Grouping: crate2 and crate3 are part of Group A, with crate2 required to be loaded first.
- **Challenge:** Safely loading the fragile crate while maintaining group order.
- **Objective:** Load crate1, crate2, and crate3, ensuring that the fragile crate is handled correctly.

3.3.3 Problem 2 - Complex Scenario (Multi-Group and Battery Management)

- **Description:** This scenario introduces two crate groups (Group A and Group B), each with a specified loading order. Battery management becomes critical.
- **Key Aspects:**
 - Four crates distributed across two groups:
 - * Group A: crate1 (heavy), crate2 (fragile).
 - * Group B: crate3 and crate4 (both light).
 - Loading order is specified within each group.
 - Both robots have limited battery (20 units), requiring efficient task planning.
- **Challenge:** Managing battery usage while maintaining the group loading order.
- **Objective:** Load all four crates while respecting group order and managing battery usage.

3.3.4 Problem 3 - Complex Scenario (Unreachable Crate Due to Battery Constraints)

- **Description:** A scenario where crate3 is located at a distance that makes it unreachable with the available battery levels of the robots.
- **Key Aspects:**
 - Four crates: three in Group A (one heavy, one fragile, one standard) and one standard crate without a group.
 - Group loading order is enforced for Group A.
 - Battery management is critical, with crate3 being unreachable with actual limited battery (20 units).
 - A recharge station is available at the loading bay.
- **Challenge:** Crate3 is unreachable because it consumes battery more than 20 units.
- **Objective:** Load crate1, crate2, crate3 and crate4 onto the conveyor belt.

3.3.5 Problem 4 - Advanced Scenario (Multiple Groups and Fragile Crates)

- **Description:** This scenario is the most complex, featuring two groups of crates, multiple fragile items, and intricate loading sequences.
- **Key Aspects:**
 - Six crates:
 - * Group A: crate1 and crate2 (one fragile).
 - * Group B: crate3, crate4, and crate5 (all fragile).
 - * Crate6: not part of any group.
 - Group loading order is specified for Group A and Group B.
 - Crate5 is located at the furthest distance, making battery management essential.

- **Challenge:** Coordinating loading across two groups with multiple fragile crates while ensuring sufficient battery for all tasks.
- **Objective:** Load all six crates while maintaining group order, properly handling fragile crates, and managing battery usage.

4 Heuristic and Pattern Design

4.1 Heuristic Function Design

4.1.1 Heuristic Objective

We define a heuristic function $h : S \rightarrow \mathbb{N}$ that estimates the remaining time required to complete the task from a given state S . Specifically, the heuristic returns a lower-bound estimate of the time needed to move all remaining crates to the loading bay, assuming ideal conditions with no delays, bottlenecks, or resource contention. This enables the planner to prioritize states closer to the goal, in which more crates have already been delivered.

4.1.2 Pseudocode

Heuristic Function $h(S)$

```
h(state):
    h_value = 0
    for each crate c not yet loaded in state:
        dist = distance(c)
        weight = weight(c)
        if weight > 50:
            time = dist * weight / 100 # requires two robots
        else:
            time_single = dist * weight / 100
            time_duo = dist * weight / 150
            time = min(time_single, time_duo)
        h_value += ceil(time)
    return h_value
```

4.1.3 Heuristic Description

This heuristic computes a lower-bound cost of reaching the goal state by estimating the minimum time required to deliver each crate to the loading bay:

- For heavy crates ($> 50\text{kg}$), two robots are required, and the time is calculated using the domain formula:

$$\text{time} = \frac{\text{distance} \times \text{weight}}{100}.$$
- For light crates, the heuristic compares the time required for single-robot and dual-robot transport and uses the faster of the two options.
- The total heuristic value is the sum of the rounded-up transport time for each remaining crate.

No loading durations are included in this heuristic, which helps maintain computational efficiency and admissibility.

4.1.4 Worked Example: Problem 0.5

Crate Configuration:

- Crate 1: 70kg, 10 units from the loading bay (heavy).
- Crate 2: 20kg, 20 units from the loading bay (light).

Heuristic Computation:

- Crate 1:

$$\text{time} = \frac{70 \times 10}{100} = 7, \quad h_1 = \lceil 7 \rceil = 7$$

- Crate 2:

$$\begin{aligned} \text{time}_{\text{single}} &= \frac{20 \times 20}{100} = 4, & \text{time}_{\text{duo}} &= \frac{20 \times 20}{150} \approx 2.67 \\ \text{time} &= 2.67, & h_2 &= \lceil 2.67 \rceil = 3 \end{aligned}$$

Total Heuristic Value:

$$h(S) = h_1 + h_2 = 7 + 3 = 10$$

4.1.5 Why the Heuristic is Useful

This heuristic:

- Is **fast to compute**, relying only on crate distance and weight.
- **Guides search efficiently** by ranking states according to remaining work.
- Encourages prioritizing crates that are closer or lighter.
- Ensures that, given two states S_1 and S_2 , if S_2 has more crates delivered, then $h(S_2) < h(S_1)$.

4.1.6 Admissibility

The heuristic is **admissible** because it never overestimates the true remaining time:

- It assumes ideal crate movement with no delays from robot scheduling or loading bay contention.
- It does not account for coordination overhead, loading times, or recharge delays.

While it may underestimate the real plan cost, it still ensures correctness in heuristic-guided optimal planners.

4.1.7 Heuristic vs. Actual Plan Comparison

In Problem 0.5, the heuristic returned a total estimate of $h(S) = 10$, while the actual plan computed by the TFD planner had a makespan of **22.06**. This gap is expected and arises from:

- **Shared resource constraints**, such as only two mover robots and one loader.
- **Sequential loading**: only one crate can be loaded at a time.
- **Robot reuse and movement delays** due to robot availability and travel paths.

Despite the numeric difference, the heuristic remains admissible and effectively guides the search by providing a lower-bound estimate.

4.1.8 Conclusion

This domain-specific heuristic effectively estimates the minimum time required to reach the goal while remaining fast and scalable. Although it does not model contention or detailed resource constraints, it aligns well with the domain metric and supports informed, admissible planning.

4.2 Pattern Definition and Symbolic Bounds

4.2.1 Pattern for Problem 0.5

The following variables represent the minimal symbolic pattern capturing the key components of the solution to Problem 0.5:

Selected Pattern Variables for Problem 0.5

- `at(robot1 loc10)` – robot1 is at the location of crate1
- `at(robot2 loc10)` – robot2 is at the location of crate1
- `crate-at(crate1 loc10)` – heavy crate that requires two robots
- `crate-at(crate2 loc20)` – light crate (could be moved single or in duo)
- `crate-loaded(crate1)` – goal state for crate1
- `crate-loaded(crate2)` – goal state for crate2

This pattern captures the essence of the actions required: reaching crates, coordinating robot positions, and achieving the loading goals.

4.2.2 Symbolic Pattern Bound for Problem 0.5

When using this pattern in a symbolic pattern database (PDB), the resulting abstract heuristic provides a lower bound on the actual cost to achieve the goal. This bound includes:

- The time to move crate1 (heavy, needs two robots) from loc10 to the loading bay
- The time to load crate1
- The time to move crate2 (light) from loc20 to the loading bay
- The time to load crate2

Estimated Pattern Bound

$$\text{Bound} = \text{move}(\text{crate1}) + \text{load}(\text{crate1}) + \text{move}(\text{crate2}) + \text{load}(\text{crate2}) \approx 22.06$$

This matches the actual makespan returned by the TFD planner for Problem 0.5, indicating that the selected pattern is compact yet highly informative for heuristic evaluation.

4.2.3 Behavior in Larger Problems

In more complex instances (e.g: Problems 2 or 3 ...), where there are more crates, extended distances, energy constraints, or ordering requirements:

- The bound will **increase** proportionally with the number of crates and movement distance.
- This pattern will **no longer be sufficient** since it omits:
 - Battery levels and recharge actions
 - Group ordering constraints
 - Shared resource contention at the loading bay

4.2.4 Generalizing the Pattern

To improve precision and maintain heuristic informativeness on larger problems, the following additions can be made to the pattern:

Generalizing the Pattern for Larger Problems

- Include `battery-level(robot1)` and `battery-level(robot2)` to track energy constraints.
- Track intermediate robot positions such as `at(robot1 loc20)`.
- Add group-specific ordering predicates: `crate-group(crateX groupA)` and `group-next(crateX crateY)`.
- Consider recharging conditions with `currently-in-loading-bay(robot)`.

These extensions increase the expressiveness of the pattern database (PDB), leading to tighter heuristic bounds at the cost of higher abstraction size.

5 Conclusion

5.1 Experimental Results

This part presents and analyzes the results of running the planner in five different warehouse scenarios, each designed to test specific constraints and extensions.

5.1.1 Problem 0.5

Crates: 2

Properties:

- Crate1: 70kg, distance = 10, group A
- Crate2: 20kg, distance = 20, group A

As shown in Figure 1, the planner efficiently completed the scenario with two crates using a simple plan. Since there were no fragile crates or battery limitations, execution required only basic movement and loading, respecting the group order of Crate2.

```
found new plan:
0.00000000: (move-empty-from-loading-bay robot2 loading-bay loc10) [1.00000000]
0.01000000: (move-empty-from-loading-bay robot1 loading-bay loc10) [1.00000000]
1.02000000: (move-crate-duo robot1 robot2 crate1 loc10 loading-bay) [7.00000000]
8.03000000: (load-crate-primary crate1) [4.00000000]
12.04000000: (move-empty-from-loading-bay robot2 loading-bay loc20) [2.00000000]
14.05000000: (move-crate-single robot2 crate2 loc20 loading-bay) [4.00000000]
18.06000000: (load-crate-secondary crate2) [4.00000000]
solution with original makespan 22.06 found (ignoring no-moving-targets-rule).
plan length: 7 step(s).
makespan   : 22.06
makespan   : 22.06
search time: 0 seconds - walltime: 0.00183105 seconds
total time: 0 seconds - walltime: 0.00912189 seconds
```

Figure 1: Plan output for Problem 0.5

5.1.2 Problem 1

Crates: 3

Properties:

- Crate1: 70kg, distance = 10.
- Crate2: 20kg, distance = 20, fragile, group A
- Crate3: 20kg, distance = 20, group A

As shown in Figure 2, the planner successfully handled a fragile crate using the secondary loader, while maintaining group order and optimizing movement with minimal plan length.

```

found new plan:
0.00000000: (move-empty-from-loading-bay robot1 loading-bay loc10) [1.00000000]
0.01000000: (move-empty-from-loading-bay robot2 loading-bay loc10) [1.00000000]
1.02000000: (move-crate-duo robot1 robot2 crate1 loc10 loading-bay) [7.00000000]
8.03000000: (load-crate-primary crate1) [4.00000000]
12.04000000: (move-empty-from-loading-bay robot1 loading-bay loc20) [2.00000000]
12.05000000: (move-empty-from-loading-bay robot2 loading-bay loc20) [2.00000000]
14.06000000: (move-crate-light-duo robot1 robot2 crate2 loc20 loading-bay) [2.66667000]
16.73667000: (load-crate-fragile-secondary crate2) [6.00000000]
22.74667000: (move-empty-from-loading-bay robot1 loading-bay loc20) [2.00000000]
24.75667000: (move-crate-single robot1 robot2 crate3 loc20 loading-bay) [4.00000000]
28.76667000: (load-crate-secondary crate3) [4.00000000]
solution with original makespan 32.7667 found (ignoring no-moving-targets-rule).
plan length: 11 step(s).
makespan : 32.7667
makespan : 32.7667
search time: 0 seconds - walltime: 0.00317407 seconds
total time: 0 seconds - walltime: 0.0114961 seconds

```

Figure 2: Plan output for Problem 1

5.1.3 Problem 2

Crates: 4

Properties:

- Crate1: 70kg, distance = 10, group A
- Crate2: 80kg, distance = 20, fragile, group A
- Crate3: 20kg, distance = 20, group B
- Crate4: 30kg, distance = 10, group B

As shown in Figure 3 fig:problem2, the planner managed two crate groups (A and B) and handled both a fragile crate and a heavy crate. The plan demonstrates careful coordination of battery usage and loader assignment to satisfy all dependencies.

```

found new plan:
0.00000000: (move-empty-from-loading-bay robot1 loading-bay loc10) [1.00000000]
0.01000000: (move-empty-from-loading-bay robot2 loading-bay loc10) [1.00000000]
1.02000000: (move-crate-duo robot1 robot2 crate1 loc10 loading-bay) [7.00000000]
8.03000000: (load-crate-primary crate1) [4.00000000]
12.04000000: (mark-ready-in-loading-bay robot1) [0.00010000]
12.05000000: (mark-ready-in-loading-bay robot2) [0.00010000]
12.06000000: (recharge robot2) [5.00000000]
17.07000000: (move-empty-from-loading-bay robot2 loading-bay loc20) [2.00000000]
17.08000000: (recharge robot1) [5.00000000]
22.09000000: (move-empty-from-loading-bay robot1 loading-bay loc20) [2.00000000]
24.10000000: (move-crate-duo robot1 robot2 crate2 loc20 loading-bay) [16.00000000]
40.11000000: (load-crate-fragile-primary crate2) [6.00000000]
46.12000000: (mark-ready-in-loading-bay robot2) [0.00010000]
46.13000000: (recharge robot2) [5.00000000]
51.14000000: (move-empty-from-loading-bay robot2 loading-bay loc20) [2.00000000]
53.15000000: (move-crate-single robot2 crate3 loc20 loading-bay) [4.00000000]
57.16000000: (load-crate-secondary crate3) [4.00000000]
61.17000000: (move-empty-from-loading-bay robot2 loading-bay loc10) [1.00000000]
62.18000000: (move-crate-single robot2 crate4 loc10 loading-bay) [3.00000000]
65.19000000: (load-crate-secondary crate4) [4.00000000]
solution with original makespan 69.19 found (ignoring no-moving-targets-rule).
plan length: 20 step(s).
makespan : 69.19
makespan : 69.19
search time: 0.01 seconds - walltime: 0.01261 seconds
total time: 0.01 seconds - walltime: 0.024477 seconds

```

Figure 3: Plan output for Problem 2

5.1.4 Problem 3

Crates: 4

Properties:

- Crate1: 70kg, distance = 20, group A
- Crate2: 80kg, distance = 20, fragile, group A
- Crate3: 60kg, distance = 30, group A
- Crate4: 30kg, distance = 10

As shown in Figure 4 fig:problem3, the planner overcame the battery limitation for Crate3 by using a relay strategy via a midpoint. The plan successfully incorporates movement splitting and recharge actions to transport the heavy crate from a distant location.

```
found new plan:
0.00000000: (move-empty-from-loading-bay robot1 loading-bay loc10) [1.00000000]
0.01000000: (move-empty-from-loading-bay robot2 loading-bay loc10) [1.00000000]
1.02000000: (move-crate-duo robot1 robot2 crate1 loc10 loading-bay) [7.00000000]
8.03000000: (load-crate-primary crate1) [4.00000000]
12.04000000: (mark-ready-in-loading-bay robot2) [0.00010000]
12.05000000: (mark-ready-in-loading-bay robot1) [0.00010000]
12.06000000: (recharge robot1) [5.00000000]
12.07000000: (recharge robot2) [5.00000000]
17.08000000: (move-empty-from-loading-bay robot2 loading-bay loc20) [2.00000000]
17.09000000: (move-empty-from-loading-bay robot1 loading-bay loc20) [2.00000000]
19.10000000: (move-crate-duo robot1 robot2 crate2 loc20 loading-bay) [16.00000000]
35.11000000: (load-crate-fragile-primary crate2) [6.00000000]
41.12000000: (mark-ready-in-loading-bay robot1) [0.00010000]
41.13000000: (mark-ready-in-loading-bay robot2) [0.00010000]
41.14000000: (recharge robot1) [5.00000000]
46.15000000: (move-empty-from-loading-bay robot1 loading-bay loc30) [3.00000000]
46.16000000: (recharge robot2) [5.00000000]
51.17000000: (move-empty-from-loading-bay robot2 loading-bay loc30) [3.00000000]
54.18000000: (move-crate-duo-relay robot1 robot2 crate3 loc30 loc15) [9.00000000]
63.19000000: (move-empty-to-loading-bay robot1 loc15 loading-bay) [1.50000000]
63.20000000: (move-empty-to-loading-bay robot2 loc15 loading-bay) [1.50000000]
64.71000000: (mark-ready-in-loading-bay robot1) [0.00010000]
64.72000000: (mark-ready-in-loading-bay robot2) [0.00010000]
64.73000000: (recharge robot1) [5.00000000]
69.74000000: (move-empty-from-loading-bay robot1 loading-bay loc15) [1.50000000]
69.75000000: (recharge robot2) [5.00000000]
74.76000000: (move-empty-from-loading-bay robot2 loading-bay loc15) [1.50000000]
76.27000000: (move-crate-duo-to-final robot1 robot2 crate3 loc15 loading-bay) [9.00000000]
85.28000000: (load-crate-primary crate3) [4.00000000]
89.29000000: (move-empty-from-loading-bay robot2 loading-bay loc10) [1.00000000]
90.30000000: (move-crate-single robot2 crate4 loc10 loading-bay) [3.00000000]
93.31000000: (load-crate-secondary crate4) [4.00000000]
solution with original makespan 97.31 found (ignoring no-moving-targets-rule).
plan length: 32 step(s).
makespan : 97.31
makespan : 97.31
search time: 0.09 seconds - walltime: 0.103951 seconds
total time: 0.11 seconds - walltime: 0.12922 seconds
```

Figure 4: Plan output for Problem 3

5.1.5 Problem 4

Crates: 6

Properties:

- Crate1: 30kg, distance = 20, group A
- Crate2: 20kg, distance = 20, fragile, group A
- Crate3: 30kg, distance = 10, fragile, group B

- Crate4: 20kg, distance = 20, fragile, group B
- Crate5: 30kg, distance = 30, fragile, group B
- Crate6: 20kg, distance = 10

As shown in Figure 5 fig:problem4, the planner executed a complex scenario involving multiple fragile crates across two groups. It effectively utilized the secondary loader for light crates and ensured group-ordering while managing extended movement and recharge needs.

```
found new plan:
0.00000000: (move-empty-from-loading-bay robot2 loading-bay loc20) [2.00000000]
0.01000000: (move-empty-from-loading-bay robot1 loading-bay loc20) [2.00000000]
2.02000000: (move-crate-light-duo robot1 robot2 crate1 loc20 loading-bay) [4.00000000]
6.03000000: (load-crate-secondary crate1) [4.00000000]
6.04000000: (move-empty-from-loading-bay robot1 loading-bay loc20) [2.00000000]
6.05000000: (move-empty-from-loading-bay robot2 loading-bay loc20) [2.00000000]
10.04000000: (move-crate-light-duo robot1 robot2 crate2 loc20 loading-bay) [2.66667000]
12.71667000: (load-crate-fragile-secondary crate2) [6.00000000]
12.72667000: (move-empty-from-loading-bay robot1 loading-bay loc10) [1.00000000]
12.73667000: (move-empty-from-loading-bay robot2 loading-bay loc10) [1.00000000]
18.72667000: (move-crate-light-duo robot1 robot2 crate3 loc10 loading-bay) [2.00000000]
20.73667000: (load-crate-fragile-secondary crate3) [6.00000000]
20.74667000: (move-empty-from-loading-bay robot2 loading-bay loc20) [2.00000000]
22.75667000: (move-empty-from-loading-bay robot1 loading-bay loc20) [2.00000000]
26.74667000: (move-crate-light-duo robot1 robot2 crate4 loc20 loading-bay) [2.66667000]
29.42334000: (load-crate-fragile-secondary crate4) [6.00000000]
29.43334000: (mark-ready-in-loading-bay robot1) [0.00010000]
29.44334000: (mark-ready-in-loading-bay robot2) [0.00010000]
29.45334000: (recharge robot1) [5.00000000]
29.46334000: (recharge robot2) [5.00000000]
34.47334000: (move-empty-from-loading-bay robot1 loading-bay loc30) [3.00000000]
34.48334000: (move-empty-from-loading-bay robot2 loading-bay loc30) [3.00000000]
37.49334000: (move-crate-light-duo robot1 robot2 crate5 loc30 loading-bay) [6.00000000]
43.50334000: (load-crate-fragile-secondary crate5) [6.00000000]
43.51334000: (move-empty-from-loading-bay robot1 loading-bay loc10) [1.00000000]
49.51334000: (move-crate-single robot1 crate6 loc10 loading-bay) [2.00000000]
51.52334000: (load-crate-secondary crate6) [4.00000000]
solution with original makespan 55.5233 found (ignoring no-moving-targets-rule).
plan length: 27 step(s).
makespan   : 55.5233
makespan   : 55.5233
search time: 0.6 seconds - walltime: 0.6096 seconds
total time: 0.62 seconds - walltime: 0.63824 seconds
```

Figure 5: Plan output for Problem 4

5.1.6 Summary of Results

The table below summarize the outcomes of the five planning problems tested using the warehouse domain. The scenarios vary in complexity by introducing different numbers of crates, distances, and handling constraints (e.g., fragile crates, group ordering, heavy crates, and battery limitations). The planner successfully found valid plans for all scenarios.

- The makespan tends to increase with the number of crates, the presence of fragile crates, and the distance from the loading bay.
- Problem 3 had the highest makespan (97.31s) due to the need for relay strategies and recharging, while Problem 0.5 was the most efficient (22.06s) with only two crates and no fragile constraints.
- Group ordering (Extension 1), battery-aware transport (Extension 3), dual loaders (Extension 2), and special actions for fragile crates (Extension 4) were all successfully handled.

Table 1: Summary of Planning Results

Problem	Crates	Max Dist	Fragile	Groups	Makespan	Plan Len.	Search Time (s)
0.5	2	20	0	A	22.06	7	0.00
1	3	20	1	A	32.77	11	0.00
2	4	20	1	A, B	69.19	20	0.01
3	4	30	1	A	97.31	32	0.09
4	6	30	5	A, B	55.52	27	0.06

5.2 Challenges Faced

During the development and testing of the warehouse planning domain, we encountered two significant challenges that required design adjustments and domain-level reasoning to resolve.

5.2.1 Battery Constraints and Inaccessible Crates (Problem 3)

In Problem 3, one of the crates (Crate3) was positioned at a distance of 30 units from the loading bay and weighed 60 kg. The battery model in our domain assigns a maximum of 20 units to each robot. According to the battery cost formula ($\text{distance} \times \text{weight} / 100$), moving this heavy crate from `loc30` to the loading bay would require **18 battery units** per robot.

However, to perform this action, each robot must first move from the loading bay to `loc30`, which consumes an additional **3 units** (based on the empty movement cost formula ($\text{distance} / 10$)). This brings the total battery requirement to **21 units** — exceeding the available 20-unit capacity.

Solution:

To resolve this, we introduced an intermediate location, or `midpoint`, between `loc30` and the loading bay. The robots first move the heavy crate to the midpoint, return to the loading bay to recharge, and then complete the second leg of the transport. This relay strategy ensures that no single action exceeds the robot's battery capacity, enabling the planner to find a valid solution for this otherwise infeasible scenario.

5.2.2 Loader Role Separation and Balanced Task Distribution

Another challenge involved enforcing the correct division of labor between the two available loaders in the warehouse: a primary loader and a secondary loader. According to the project requirements:

- **Primary Loader (Loader 1)** is responsible for loading all **heavy crates**, including both fragile and non-fragile ones.
- **Secondary Loader (Loader 2)** is designated to load only **non-heavy crates**, which may be either fragile or not.

Initial Issue: In our earlier implementation, the primary loader was capable of loading all types of crates (including light ones), while the secondary loader was limited.

As a result, the planner always assigned crate-loading tasks to Loader 1 — even when the crate was light or fragile and could have been handled by Loader 2.

This behavior violated the intended separation of responsibilities and led to unrealistic and unbalanced plan outputs where Loader 2 was unused.

Refined Design:

To resolve this, we restructured the domain actions as follows:

- `load-crate-primary` and `load-crate-fragile-primary` were updated to include the condition (`is-heavy ?c`), ensuring the primary loader is used **exclusively for heavy crates**.
- `load-crate-secondary` and `load-crate-fragile-secondary` include the condition (`not (is-heavy ?c)`), guaranteeing that only non-heavy crates are passed to the secondary loader.

As a result, the planner started assigning light and fragile-light crates to Loader 2, while Loader 1 was used only to handle heavy crates. This created a more realistic, efficient, and specification-compliant distribution of crate handling responsibilities between the two loaders.

5.3 Possible Improvements

5.3.1 Generalizing Relay Strategy for All Crates

In Problem 3, we introduced a relay strategy using an intermediate `midpoint` location to transport a distant heavy crate in two stages. This involved splitting the task into two coordinated actions: one to move the crate from its origin to the midpoint, and another from the midpoint to the loading bay after the robots recharged. This mechanism resolved the issue of insufficient battery capacity for long-distance, heavy crate movement.

Limitation: In the current domain, this relay mechanism is implemented only for heavy crates, as those were the most urgent cases (e.g., `Crate3` in Problem 3). We created dedicated actions:

- `move-crate-duo-relay`
- `move-crate-duo-to-final`

However, no equivalent actions exist for light crates. This design decision was made to keep the domain manageable in size and complexity during the project time frame.

Improvement Suggestion: To improve flexibility, the relay concept could be extended to cover **all crates types**, not just heavy ones. For example:

- `move-crate-single-relay`
- `move-crate-single-to-final`
- `move-crate-light-duo-relay`
- `move-crate-light-duo-to-final`

This would allow robots to transport light crates over long distances without battery constraint violations, further increasing the domain's scalability and realism. Although adding such actions would increase the domain file's length, it would generalize the energy management strategy to all planning scenarios.